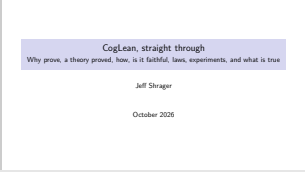
CogLean, straight through

Why prove, a theory proved, how, is it faithful, laws, experiments, and what is true

Jeff Shrager

October 2026

CogLean, straight through

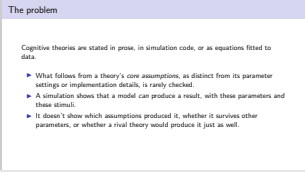


Today I want to tell you about CogLean, a project to build a machine-checked library of cognitive theory, in the spirit of Mathlib for mathematics. I'll go in eight parts: why prove things about cognitive theories at all; one theory we've proved things about; what the proofs actually look like; how we check that the formal version says what the theory says; how to generalize beyond one model; laws of memory, and a matrix comparing families of models; experiments derived from those laws; and finally, what can count as true when the theories compete.

The problem

Cognitive theories are stated in prose, in simulation code, or as equations fitted to data.

- What follows from a theory's *core assumptions*, as distinct from its parameter settings or implementation details, is rarely checked.
- A simulation shows that a model *can* produce a result, with these parameters and these stimuli.
- It doesn't show which assumptions produced it, whether it survives other parameters, or whether a rival theory would produce it just as well.



Part 1, why: what simulation leaves open.

Most of our theories live in prose, in simulation code, or as equations fitted to data. When we evaluate them, we usually run a model with particular parameters on particular stimuli and compare the output with data. That tells us the theory can produce a result. It doesn't tell us which of the theory's assumptions produced it, whether it would survive a different choice of parameters, or whether a rival theory would produce it just as well. Those are exactly the questions we care about when we compare theories, and they are rarely answered.

Three kinds of evidence a theory can give

One run	**Every run**	**Every model satisfying the postulates**
one model, one parameter setting	one model, all its executions	all models the theory allows
the theory *can* do it	that model *always* does it	the postulates *entail* it
simulation	model checking, sweeps	proof

CogLean is built around the third kind, and keeps the other two in the same language.

Three kinds of evidence a theory can give

Three kinds of evidence a theory can give

One run	Every run	Every model satisfying the postulates
one model, one parameter setting	one model, all its executions	all models the theory allows
the theory *can* do it	that model *always* does it	the postulates *entail* it
simulation	model checking, sweeps	proof

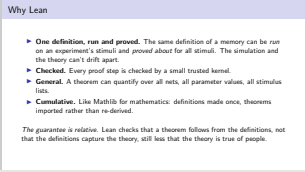
CogLean is built around the third kind, and keeps the other two in the same language.

It helps to separate three kinds of evidence. One run: one model, one parameter setting; the theory can do it. Every run: one model in all its executions, by exhaustive search or parameter sweeps; that model always does it. And every model the theory allows: the postulates entail it. That third kind is a proof, and it's what CogLean is built around. A proof tells you which assumptions matter, because it uses some postulates and not others, and it distinguishes what a theory can do from what it must do. Simulation stays essential for exploring and fitting, and experiments with people still decide whether the assumptions are right.

Why Lean

- **One definition, run and proved.** The same definition of a memory can be *run* on an experiment's stimuli and *proved about* for all stimuli. The simulation and the theory can't drift apart.
- **Checked.** Every proof step is checked by a small trusted kernel.
- **General.** A theorem can quantify over all nets, all parameter values, all stimulus lists.
- **Cumulative.** Like Mathlib for mathematics: definitions made once, theorems imported rather than re-derived.

The guarantee is relative. Lean checks that a theorem follows from the definitions, not that the definitions capture the theory, still less that the theory is true of people.



We use Lean, which is both a programming language and a proof checker. That matters because the same definition, say of a discrimination net, can be run on an experiment's stimuli and proved about for all stimuli, so the simulation and the theory can't drift apart. Every proof step is checked by a small trusted kernel. Theorems can quantify over all parameter values. And the library is cumulative: definitions are made once and reused. One caution, which I'll come back to: Lean checks that theorems follow from the definitions, not that the definitions capture the theory.

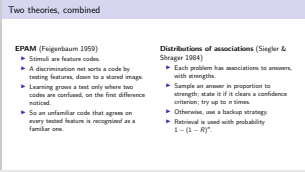
Two theories, combined

EPAM (Feigenbaum 1959)

- Stimuli are feature codes.
- A discrimination net sorts a code by testing features, down to a stored image.
- Learning grows a test only where two codes are confused, on the first difference noticed.
- So an unfamiliar code that agrees on every tested feature is *recognized as* a familiar one.

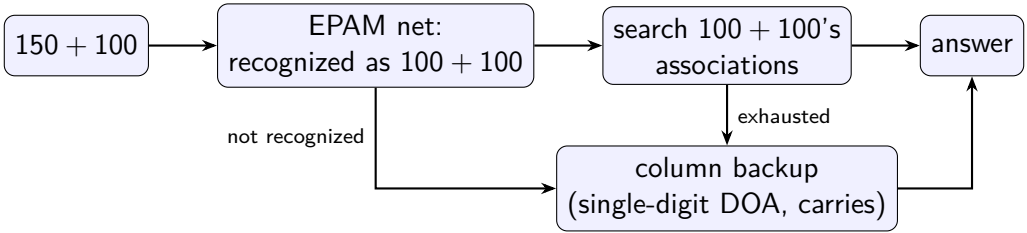
Distributions of associations (Siegler & Shrager 1984)

- Each problem has associations to answers, with strengths.
- Sample an answer in proportion to strength; state it if it clears a confidence criterion; try up to n times.
- Otherwise, use a backup strategy.
- Retrieval is used with probability $1 - (1 - R)^n$.



Part 2, a theory proved: two classic models combined into one of multi-digit addition. Here are two classic theories. EPAM, Feigenbaum's model of verbal learning: stimuli are feature codes, sorted through a discrimination net of feature tests, and the net grows a new test only where two codes get confused. So an unfamiliar code that matches a familiar one on every tested feature gets recognized as that familiar one. And the distributions-of-associations model of strategy choice, which Bob Siegler and I proposed: each problem has associations to answers; you sample an answer in proportion to its strength, state it if it clears a confidence criterion, try a few times, and otherwise fall back on a backup strategy such as counting.

The multi-digit model



- Every postulate is a parameter: the encoding, the net, the strengths, the criterion, the search length, the backup.
- 18 predictions, each proved for every choice of them.

The multi-digit model



- Every postulate is a parameter: the encoding, the net, the strengths, the criterion, the search length, the backup.
- 18 predictions, each proved for every choice of them.

Combine them and you get a model of three-digit addition. A problem like 150 plus 100 is first recognized by an EPAM net, perhaps as the familiar 100 plus 100. Then retrieval searches the recognized problem's associations. If nothing is recognized, or retrieval comes up empty, the child works it out column by column, where each column is itself a single-digit trial of retrieval or backup, with carries. Every postulate here is a parameter: the encoding, the net, the strengths, the criterion, the search length, the backup. The model yields eighteen predictions, each proved for every choice of those parameters.

Confident errors

Let a familiar problem q have *sharp* associations (only its correct sum clears the criterion), and let x, with a different sum, be recognized as q. Then:

1. on q, whole-problem retrieval never errs;
2. on x, every whole-problem answer is wrong;
3. x uses the backup exactly as rarely as q does: $(1 - R_q)^n$.

In DOA alone, errors and backup use rise together. Confusion breaks the tie: a retrieved, wrong answer with no extra backup use.

Confident errors

Let a familiar problem q have *sharp* associations (only its correct sum clears the criterion), and let x, with a different sum, be recognized as q. Then:

1. on q, whole-problem retrieval never errs;
2. on x, every whole-problem answer is wrong;
3. x uses the backup exactly as rarely as q does: $(1 - R_q)^a$.

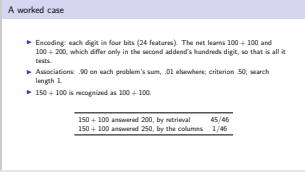
In DOA alone, errors and backup use rise together. Confusion breaks the tie: a retrieved, wrong answer with no extra backup use.

Here is the prediction neither theory makes alone. Suppose a familiar problem q has sharp associations, meaning only its correct sum clears the criterion, and a stranger x with a different sum is recognized as q. Then q is never answered wrongly by retrieval; every retrieved answer to x is wrong; and x falls back on the columns exactly as rarely as q does. In the associations model alone, errors and backup use rise and fall together. Confusion breaks that tie: you get a retrieved, wrong answer with no extra backup use. Notice that I don't say fast: the model has no timing, so that would need more.

A worked case

- Encoding: each digit in four bits (24 features). The net learns $100 + 100$ and $100 + 200$, which differ only in the second addend's hundreds digit, so that is all it tests.
- Associations: .90 on each problem's sum, .01 elsewhere; criterion .50; search length 1.
- $150 + 100$ is recognized as $100 + 100$.

$150 + 100$ answered 200, by retrieval	45/46
$150 + 100$ answered 250, by the columns	1/46



To see it concretely: encode each digit in four bits. A net that has learned 100 plus 100 and 100 plus 200 only needs to test the second addend's hundreds digit to tell them apart. So 150 plus 100 slips through as 100 plus 100. With these strengths and a single search, it's answered 200 by retrieval with probability 45 over 46, and correctly, by the columns, with probability 1 over 46. These numbers are just an illustration, computed exactly; the structure, confident errors, holds for every choice of parameters.

In words: Confident errors: the proof is a composition

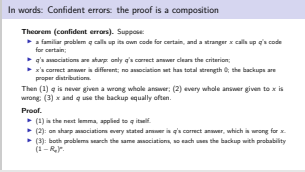
Theorem (confident errors). Suppose:

- a familiar problem q calls up its own code for certain, and a stranger x calls up q's code for certain;
- q's associations are *sharp*: only q's correct answer clears the criterion;
- x's correct answer is different; no association set has total strength 0; the backups are proper distributions.

Then (1) q is never given a wrong whole answer; (2) every whole answer given to x is wrong; (3) x and q use the backup equally often.

Proof.

- (1) is the next lemma, applied to q itself.
- (2): on sharp associations every stated answer is q's correct answer, which is wrong for x.
- (3): both problems search the same associations, so each uses the backup with probability $(1 - R_q)^n$.


In words: Confident errors: the proof is a composition

Theorem (confident errors). Suppose:

- x familiar problem q calls up its own code for certain, and a stranger x calls up q's code for certain;
- q's associations are *sharp*: only q's correct answer clears the criterion;
- x's correct answer is different; no association set has total strength 0; the backups are proper distributions.

Then (1) q is never given a wrong whole answer; (2) every whole answer given to x is wrong; (3) x and q use the backup equally often.

Proof.

- (1) is the next lemma, applied to q itself.
- (2): on sharp associations every stated answer is q's correct answer, which is wrong for x.
- (3): both problems search the same associations, so each uses the backup with probability $(1 - R_q)^n$.

Part 3, how: the proof in words, then the actual Lean.

So how is that proved? Here is the proof in words first. The hypotheses are the ones I described: both problems are recognized for certain, q's associations are sharp, the correct answers differ, and the strengths and backups are well behaved. The proof has three parts, and each is a lemma we had already proved. The first is the neighbour's-answer lemma, applied to q itself. The second says that on sharp associations every stated answer is q's correct answer, which is wrong for x. And the third: both problems search the same associations, so each uses the backup with the same probability, one minus R to the n.

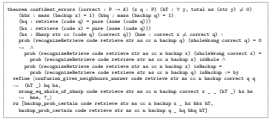
Confident errors: the proof is a composition

```
theorem confident_errors (correct : P → A) (x q : P) (hT : ∀ y, total as (str y) ≠ 0)
    (hbx : mass (backup x) = 1) (hbq : mass (backup q) = 1)
    (hq : retrieve (code q) = pure (some (code q)))
    (hx : retrieve (code x) = pure (some (code q)))
    (hs : Sharp str cc (code q) (correct q)) (hne : correct x ≠ correct q) :
    prob (recognizeRetrieve code retrieve str as cc n backup q) (wholeWrong correct q) = 0
    ↪ ∧
      prob (recognizeRetrieve code retrieve str as cc n backup x) (wholeWrong correct x) =
        prob (recognizeRetrieve code retrieve str as cc n backup x) isWhole ∧
      prob (recognizeRetrieve code retrieve str as cc n backup x) isBackup =
        prob (recognizeRetrieve code retrieve str as cc n backup q) isBackup := by
  refine ⟨confusion_gives_neighbours_answer code retrieve str as cc n backup correct q q
  ↪ (hT _) hq hs,
    wrong_eq_whole_of_sharp code retrieve str as cc n backup correct x _ _ (hT _) hx hs
    ↪ hne, ?_⟩
  rw [backup_prob_certain code retrieve str as cc n backup x _ hx hbx hT,
    backup_prob_certain code retrieve str as cc n backup q _ hq hbq hT]
```

CogLean, straight through

Confident errors: the proof is a composition

Confident errors: the proof is a composition



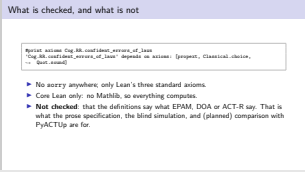
```
theorem confident_errors (correct : P → A) (q q : P) (hT : ∀ y, total as (str y) ≠ 0)
    (hbin : mass (backup x) = 1) (hdeg : mass (backup q) = 1)
    (hq : retrieves (code q) = pure (some (code q)))
    (hx : retrieves (code x) = pure (some (code q)))
    (hm : #hamp str (code q) (correct q)) (hne : correct x ≠ correct q) :
  prob (recognizesRetrieve code retrieve str as cc x backup q) (fallsBelow correct q) = 0 :=
  prob (recognizesRetrieve code retrieve str as cc x backup q) (fallsBelow correct q)
      = prob (recognizesRetrieve code retrieve str as cc x backup x) (fallsBelow correct q) :=
        prob (recognizesRetrieve code retrieve str as cc x backup x) toWhole ▸
        prob (recognizesRetrieve code retrieve str as cc x backup x) toBackup ▸
          prob (recognizesRetrieve code retrieve str as cc x backup q) toBackup := by
  refine (confusion_gives_neighbours_nearest code retrieve str as cc x backup correct q q
    (by _) hq hx
    (wrong_eq_choice_of_nhamp code retrieve str as cc x backup correct x _ (by _) hx hm
      hne.symm)
    (backup_prob_certain code retrieve str as cc x backup x _ hx hhx hT)
    (backup_prob_certain code retrieve str as cc x backup q _ hq hhq hT)
```

And here is the actual Lean. You don't need to read it in detail. The statement at the top is the theorem I just described, and the proof is the last five lines, which assemble those three lemmas. It's that short because the real work sits in general lemmas underneath: about probability distributions, about the search process, about nets. That's the point I want to make with this slide. In a library, a behavioral theorem is assembled from reusable parts, the way Mathlib assembles algebra.

What is checked, and what is not

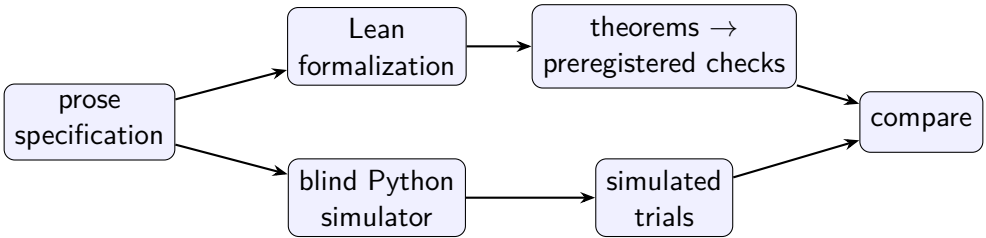
```
#print axioms Cog.RR.confident_errors_of_laws
'Cog.RR.confident_errors_of_laws' depends on axioms: [propext, Classical.choice,
↪  Quot.sound]
```

- No `sorry` anywhere; only Lean's three standard axioms.
- Core Lean only: no Mathlib, so everything computes.
- **Not checked**: that the definitions say what EPAM, DOA or ACT-R say. That is what the prose specification, the blind simulation, and (planned) comparison with PyACTUp are for.



What does all this guarantee? Lean will tell you exactly which axioms a theorem rests on. Here it's only Lean's three standard ones, with no unproved steps anywhere. We use core Lean without Mathlib, so everything also computes. What Lean can't check is whether the definitions say what EPAM, or the associations model, or ACT-R actually say. That is a different kind of question, and the next part is about how we tested it.

The idea: two independent readings



N-version programming (Avižienis 1985), applied to a theory. A disagreement means one of three things: the prose is ambiguous, the formalization drifted, or the simulator has a bug.

The idea: two independent readings



N-version programming (Avižienis 1985), applied to a theory. A disagreement means one of three things: the prose is ambiguous, the formalization drifted, or the simulator has a bug.

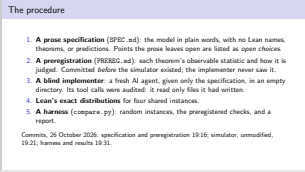
Part 4, is it faithful: a blind simulation against preregistered predictions.

We borrowed an idea from fault-tolerant software, called N-version programming: build two independent implementations of one specification, and look at where they disagree. Here, one reading of the prose is the Lean formalization, whose theorems give predictions. The other is a Python simulator written without seeing any of the Lean. If they disagree, then either the prose is ambiguous, the formalization drifted, or the simulator has a bug, and in every case we learn something.

The procedure

1. **A prose specification** (`SPEC.md`): the model in plain words, with no Lean names, theorems, or predictions. Points the prose leaves open are listed as *open choices*.
2. **A preregistration** (`PREREG.md`): each theorem's observable statistic and how it is judged. Committed *before* the simulator existed; the implementer never saw it.
3. **A blind implementer**: a fresh AI agent, given only the specification, in an empty directory. Its tool calls were audited: it read only files it had written.
4. **Lean's exact distributions** for four shared instances.
5. **A harness** (`compare.py`): random instances, the preregistered checks, and a report.

Commits, 26 October 2026: specification and preregistration 19:16; simulator, unmodified, 19:21; harness and results 19:31.



Concretely: we wrote a prose specification with no Lean in it, and we listed explicitly the points the prose leaves open. We preregistered the predictions, committing them before the simulator existed. Then a fresh AI agent, given only the prose, wrote the simulator in an empty directory, and we audited its tool use to check that it read nothing else. Lean exported exact probability distributions for some shared cases, and a harness compared everything. The commit timestamps show the order: the specification and preregistration first, the simulator five minutes later, the results ten minutes after that.

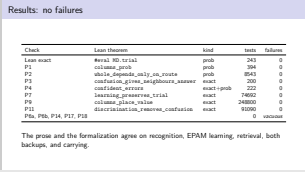
Results: no failures

Check	Lean theorem	kind	tests	failures
Lean exact	`#eval MD.trial`	prob	243	0
P1	`columns_prob`	prob	394	0
P2	`whole_depends_only_on_route`	prob	8543	0
P3	`confusion_gives_neighbours_answer`	exact	200	0
P4	`confident_errors`	exact+prob	222	0
P7	`learning_preserves_trial`	exact	74692	0
P9	`columns_place_value`	exact	248800	0
P11	`discrimination_removes_confusion`	exact	91090	0
P6a, P6b, P14, P17, P18			0	*vacuous*

The prose and the formalization agree on recognition, EPAM learning, retrieval, both backups, and carrying.

CogLean, straight through

Results: no failures



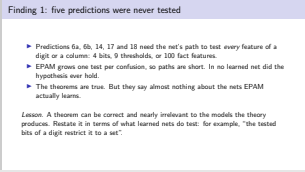
The result: no failures. 243 outcome frequencies matched Lean's exact values, and every applicable prediction passed, some of them over tens of thousands of trials. So the prose and the formalization agree on recognition, learning, retrieval, both backups and carrying. But look at the bottom row: five predictions were vacuous.

Finding 1: five predictions were never tested

- Predictions 6a, 6b, 14, 17 and 18 need the net's path to test *every* feature of a digit or a column: 4 bits, 9 thresholds, or 100 fact features.
- EPAM grows one test per confusion, so paths are short. In no learned net did the hypothesis ever hold.
- The theorems are true. But they say almost nothing about the nets EPAM actually learns.

Lesson. A theorem can be correct and nearly irrelevant to the models the theory produces. Restate it in terms of what learned nets do test: for example, "the tested bits of a digit restrict it to a set".

Finding 1: five predictions were never tested



That is the most interesting finding. Those five predictions needed the net to test every feature of a digit. But EPAM grows only one test per confusion, so in no learned net did that ever happen. The theorems are true, and yet they say almost nothing about the nets EPAM actually produces. That's a general lesson for proving things about cognitive models: the hypotheses that make a proof go through can quietly exclude the cases you care about. Without a rule that reports vacuous checks as vacuous, these would have looked like five passes.

Finding 3: a prediction depends on an open choice

The predicted learning sequence: a confident error, then column work, then fluent correct retrieval.

O2: new problem starts with	discriminated	canonical	settles on correct	mixed	trapped on error
no associations	48	40	7	1	0
only the correct answer	61	28	33	0	0
a copy of the neighbour's	44	0	2	35	7

- With no associations, the predicted sequence appears (40 of 48).
- Starting with the correct answer, the column phase is often skipped.
- Copying the neighbour's associations perpetuates the error.

Another finding concerns learning. The model predicts a sequence: a confident error, then column work after feedback, then fluent, correct retrieval. But the model doesn't say what associations a newly discriminated problem starts with; that was one of the open choices we declared. And the sequence depends on it. Starting with no associations, the predicted sequence appears in 40 of 48 cases. Starting with only the correct answer, the column phase is often skipped. Copying the neighbour's associations perpetuates the error. So a substantive question about the theory has become sharp.

Finding 3: a prediction depends on an open choice

The predicted learning sequence: a confident error, then column work, then fluent correct retrieval.

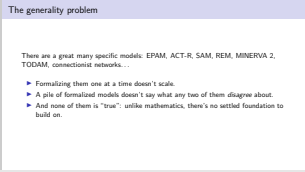
O2: new problem starts with	discriminated	canonical	settles on correct	mixed	trapped on error
no associations	48	40	7	1	0
only the correct answer	61	28	33	0	0
a copy of the neighbour's	44	0	2	35	7

- With no associations, the predicted sequence appears (40 of 48).
- Starting with the correct answer, the column phase is often skipped.
- Copying the neighbour's associations perpetuates the error.

The generality problem

There are a great many specific models: EPAM, ACT-R, SAM, REM, MINERVA 2, TODAM, connectionist networks...

- Formalizing them one at a time doesn't scale.
- A pile of formalized models doesn't say what any two of them *disagree* about.
- And none of them is "true": unlike mathematics, there's no settled foundation to build on.



Part 5, generalizing: memory as a plug-in, and a composition reused across tasks.
Part five. Everything so far is about one model. But there are a great many models, such as EPAM, ACT-R, SAM, REM, MINERVA and connectionist networks, and formalizing them one at a time doesn't scale. Worse, a pile of formalized models doesn't tell you what any two of them disagree about. And unlike mathematics, none of them is simply true. So the question is how a general library should be organized.

The same problems, two memories

	EPAM	ACT-R
$150 + 100$ answered 200 (the neighbour's sum)	45/46	225/529
$150 + 100$ answered 300 (differs on a tested digit)	0	225/1058
$150 + 100$ answered correctly	1/46	383/1058
errors on the practiced $100 + 100$	0	225/782
the same, after also studying $100 + 300$	0	3375/5842

ACT-R: confusion is graded, nothing is retrieved for certain, and new practice interferes with old.

The same problems, two memories

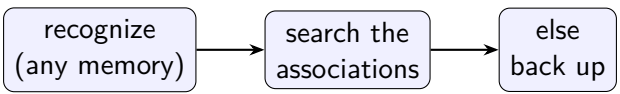
	EPAM	ACT-R
$150 \div 100$ answered 200 (the neighbour's sum)	45/46	225/529
$150 \div 100$ answered 300 (differs on a tested digit)	0	225/1058
$150 \div 100$ answered correctly	1/46	361/1058
errors on the practiced $100 \div 100$	0	225/782
the same, after also studying $100 \div 300$	0	3375/5842

ACT-R: confusion is graded, nothing is retrieved for certain, and new practice interferes with old.

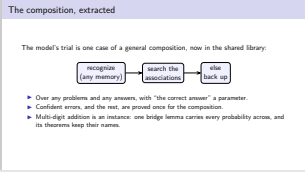
The first step was to make memory a parameter. Recognition is now an interface: memory takes a probe and returns a probability distribution over stored items. EPAM's net is one memory. ACT-R's declarative memory is another, and in the case of decay one and temperature one we can compute it exactly. Run the same problems through both, and they diverge where you'd hope: with ACT-R, confusion is graded, even a practiced problem sometimes errs, and new practice interferes with old. EPAM rules out all three.

The composition, extracted

The model's trial is one case of a general composition, now in the shared library:



- Over any problems and any answers, with "the correct answer" a parameter.
- Confident errors, and the rest, are proved once for the composition.
- Multi-digit addition is an instance: one bridge lemma carries every probability across, and its theorems keep their names.



The second step was to notice that the multi-digit trial is one case of a general composition: recognize with any memory, search the recognized item's associations, else back up. We moved that into the shared library, over any problems and any answers, and proved confident errors there, once. Multi-digit addition is now just an instance: one bridge lemma carries every probability across, and all its theorems keep their names.

A second task: naming people

Faces are codes; familiar Ann (000) and Bob (110); stranger Carl (100), one feature from each. The backup is "don't know".

	EPAM	ACT-R
Carl named Bob	45/46	225/391
Carl named Ann	0	225/782
Carl: "don't know"	1/46	107/782
Bob: "don't know"	1/46	229/2254
Bob named Ann	0	225/2254

With EPAM, *confident misnaming* is inherited from the composition's theorem, for every search length. No arithmetic anywhere.

A second task: naming people

Faces are codes; familiar Ann (000) and Bob (110); stranger Carl (100), one feature from each. The backup is "don't know".

	EPAM	ACT-R
Carl named Bob	45/46	225/391
Carl named Ann	0	225/782
Carl: "don't know"	1/46	107/782
Bob: "don't know"	1/46	229/2254
Bob named Ann	0	225/2254

With EPAM, *confident misnaming* is inherited from the composition's theorem, for every search length. No arithmetic anywhere.

To show it isn't about arithmetic, here is a second task through the same composition: naming people. Faces are codes; Ann and Bob are familiar; Carl is a stranger who resembles both. With EPAM's memory, the confident-errors theorem is simply inherited: Carl is confidently misnamed Bob, and he says don't know exactly as rarely as Bob himself does. With ACT-R's memory, the misnaming is graded: Carl can be taken for either familiar person, and he says don't know more often than Bob.

Six laws

Certainty	An item just studied is retrieved by itself with probability 1.
Stability	Studying z leaves the chance that y retrieves itself unchanged.
Order-freedom	Swapping the last two study events leaves retrieval unchanged, for every probe.
Recency	Of two new items studied last, a probe equally similar to both retrieves the later more often.
Stored-only	Nothing is retrieved that was never studied.
Determinacy	Every retrieval has a single outcome.

Six laws

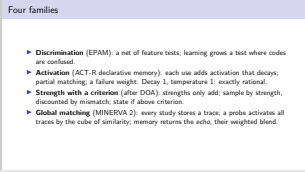
Certainty	An item just studied is retrieved by itself with probability 1.
Stability	Studying x leaves the chance that y retrieves itself unchanged.
Order-freedom	Swapping the last two study events leaves retrieval unchanged, for every probe.
Recency	Of two new items studied last, a probe equally similar to both retrieves the later more often.
Stored-only	Nothing is retrieved that was never studied.
Determinacy	Every retrieval has a single outcome.

Part 6, laws: compare families of models by the laws they have.

Part six. To compare memories in general, we need a vocabulary that doesn't mention any mechanism. These are laws: small properties a memory may or may not have. Certainty: something just studied is retrieved for certain. Stability: studying one thing doesn't change another's retrieval. Order-freedom; recency; stored-only, meaning nothing unstudied is ever retrieved; and determinacy, a single outcome. Predictions are then proved from laws, not from particular models.

Four families

- **Discrimination** (EPAM): a net of feature tests; learning grows a test where codes are confused.
- **Activation** (ACT-R declarative memory): each use adds activation that decays; partial matching; a failure weight. Decay 1, temperature 1: exactly rational.
- **Strength with a criterion** (after DOA): strengths only add; sample by strength, discounted by mismatch; state if above criterion.
- **Global matching** (MINERVA 2): every study stores a trace; a probe activates all traces by the cube of similarity; memory returns the *echo*, their weighted blend.

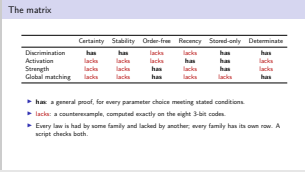


And instead of individual models, we formalize families of models, each once, with its parameters free. Discrimination nets, like EPAM. Activation memory, like ACT-R's. Strength memory with a criterion, after the associations model. And global matching, MINERVA 2, where every study leaves a trace and a probe returns a blend of all of them. A published model is then a choice of parameters within its family.

The matrix

	Certainty	Stability	Order-free	Recency	Stored-only	Determinate
Discrimination	**has**	**has**	lacks	lacks	**has**	**has**
Activation	lacks	lacks	lacks	**has**	**has**	lacks
Strength	lacks	lacks	**has**	lacks	**has**	lacks
Global matching	lacks	lacks	**has**	lacks	lacks	**has**

- **has**: a general proof, for every parameter choice meeting stated conditions.
- lacks: a counterexample, computed exactly on the eight 3-bit codes.
- Every law is had by some family and lacked by another; every family has its own row. A script checks both.



Put them together and you get a matrix. Every cell is a theorem: has means a general proof, for every parameter setting that meets stated conditions; lacks means a counterexample. Every law separates some families, and every family has its own row. I'll be candid: some of these laws are stated too narrowly, all-or-none where they should be graded, and fixing that is the next piece of work. But it's the structure I want you to see.

The laws were tested, and failed once

- The first four laws were chosen with three families in view.
- Global matching was added as a test. On those four laws, it had *exactly* the row of strength memory.
- So the laws missed something. What is distinctive about global matching? Its output is a blend.
- Study 110, 101, 011; probe with 111 (their prototype, never studied): MINERVA returns 111.
- Hence the stored-only law, which separates them.

Rule: a law enters only with a witness on each side.

The laws were tested, and failed once

- The first four laws were chosen with three families in view.
- Global matching was added as a test. On those four laws, it had *exactly* the row of strength memory.
- So the laws missed something. What is distinctive about global matching? Its output is a blend.
- Study 110, 101, 011; probe with 111 (their prototype, never studied): MINERVA returns 111.
- Hence the stored-only law, which separates them.

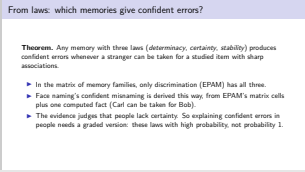
Rule: a law enters only with a witness on each side.

We also test the laws themselves. The first four were chosen with three families in mind. When we added global matching as a test, it looked exactly like strength memory on all four. So the laws were missing something, and what's distinctive about global matching is that it returns a blend. Study three variants of a pattern, probe with their prototype, and MINERVA returns the prototype, which it never saw. That forced the stored-only law. The rule now is that a law enters the library only with a family on each side.

From laws: which memories give confident errors?

Theorem. Any memory with three laws (*determinacy*, *certainty*, *stability*) produces confident errors whenever a stranger can be taken for a studied item with sharp associations.

- In the matrix of memory families, only discrimination (EPAM) has all three.
- Face naming's confident misnaming is derived this way, from EPAM's matrix cells plus one computed fact (Carl can be taken for Bob).
- The evidence judges that people lack certainty. So explaining confident errors in people needs a graded version: these laws with high probability, not probability 1.



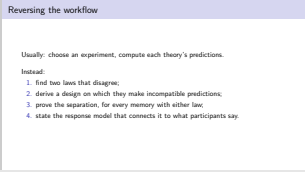
Here's where the two halves meet. Confident errors can be derived from three laws: any memory with determinacy, certainty and stability produces them, whenever a stranger can be taken for a studied item. So the matrix answers the question of which memory families produce confident errors: only discrimination. And since the evidence suggests people lack certainty, explaining confident errors in people will need a graded version of this theorem.

Reversing the workflow

Usually: choose an experiment, compute each theory's predictions.

Instead:

1. find two laws that disagree;
2. derive a design on which they make incompatible predictions;
3. prove the separation, for every memory with either law;
4. state the response model that connects it to what participants say.



Part 7, experiments from laws: start from a disagreement, derive the design, prove the separation.

Part seven. Laws let us reverse the usual workflow. Instead of choosing an experiment and computing what each model predicts, we start from two laws that disagree, derive a design on which they make incompatible predictions, prove the separation for every memory with either law, and then state the response model that connects it to what participants actually say.

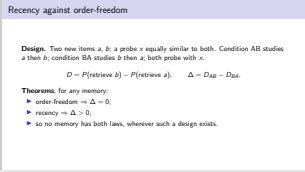
Recency against order-freedom

Design. Two new items a, b; a probe x equally similar to both. Condition AB studies a then b; condition BA studies b then a; both probe with x.

$$D = P(\text{retrieve } b) - P(\text{retrieve } a), \qquad \Delta = D_{AB} - D_{BA}.$$

Theorems, for any memory:

- order-freedom $\Rightarrow \Delta = 0$;
- recency $\Rightarrow \Delta > 0$;
- so no memory has both laws, wherever such a design exists.



Recency and order-freedom disagree. Take two new items, A and B, and a probe equally similar to both. Study A then B, or B then A, and then probe. Delta measures how much more B is favoured after AB than after BA. From the laws alone: order-freedom gives a delta of zero, and recency gives a delta greater than zero, so no memory can have both laws. These proofs never mention EPAM, or ACT-R, or any parameter.

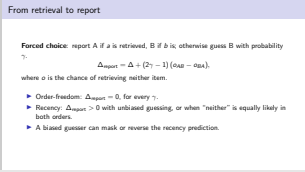
From retrieval to report

Forced choice: report A if a is retrieved, B if b is; otherwise guess B with probability γ.

$$\Delta_{\text{report}} = \Delta + (2\gamma - 1)\,(o_{AB} - o_{BA}),$$

where o is the chance of retrieving neither item.

- Order-freedom: $\Delta_{\text{report}} = 0$, for every γ.
- Recency: $\Delta_{\text{report}} > 0$ with unbiased guessing, or when "neither" is equally likely in both orders.
- A biased guesser can mask or reverse the recency prediction.



But participants give reports, not retrievals. So we add a response model: forced choice, guessing B with probability gamma when neither item comes to mind. Then the reported effect is delta plus a bias term. Order-freedom still predicts zero, whatever the bias. But recency's prediction survives only if guessing is unbiased, or if neither item is equally likely to come to mind in both orders. The bridge from theory to data comes with its own condition, and the theorem states it.

The same design, four memories

Memory	Δ, no prior item	Δ, after 000	why
ACT-R	$10/17$		recency, so $\Delta > 0$
Strength	0	0	order-free
MINERVA	0	0	order-free
EPAM	0	-2	neither law: computed

- ACT-R's reported effect is $10/17$ for every guessing bias: both orders leave the same chance of "neither".
- EPAM shows the most extreme *primacy* possible. After 000, whichever item is learned first is discriminated from it, and the probe always retrieves that one. Its order effects come from the shape of the net, not from time.

The same design, four memories

Memory	Δ, no prior item	Δ, after 000	why
ACT-R	10/17		recency, so $\Delta > 0$
Strength	0	0	order-free
MINERVA	0	0	order-free
EPAM	0	−2	neither law: computed

- ACT-R's reported effect is 10/17 for every guessing bias: both orders leave the same chance of "neither".
- EPAM shows the most extreme *primacy* possible. After 000, whichever item is learned first is discriminated from it, and the probe always retrieves that one. Its order effects come from the shape of the net, not from time.

Here is the same design run with four memories. ACT-R shows recency, ten seventeenths, and that holds for any guessing bias. Strength memory and MINERVA show no effect, as their order-freedom requires. EPAM has neither law, and it surprises us: with one prior item, it shows the most extreme primacy possible. Whichever item was learned first always wins, because of the shape of the net, not anything about time.

A related approach: SFT-GMA

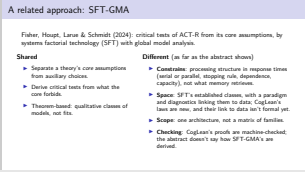
Fisher, Houpt, Larue & Schmidt (2024): critical tests of ACT-R from its core assumptions, by systems factorial technology (SFT) with global model analysis.

Shared

- Separate a theory's *core* assumptions from auxiliary choices.
- Derive critical tests from what the core forbids.
- Theorem-based: qualitative classes of models, not fits.

Different (as far as the abstract shows)

- **Constrains**: processing structure in response times (serial or parallel, stopping rule, dependence, capacity), not what memory retrieves.
- **Space**: SFT's established classes, with a paradigm and diagnostics linking them to data; CogLean's laws are new, and their link to data isn't formal yet.
- **Scope**: one architecture, not a matrix of families.
- **Checking**: CogLean's proofs are machine-checked; the abstract doesn't say how SFT-GMA's are derived.



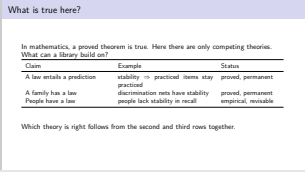
We're not alone in wanting this. Fisher, Houpt, Larue and Schmidt, at MathPsych and ICCM last year, derived critical tests of ACT-R from its core assumptions, using systems factorial technology with global model analysis; ACT-R turns out to rule out most parallel models of perceptual processing. Like us, they separate core from auxiliary assumptions. From their abstract, the differences: they constrain processing structure seen in response times, we constrain what memory retrieves and learns; their model space comes with its own paradigm and diagnostics, so its link to data is already there, and ours isn't yet; they analyze one architecture, we compare families; and our results are machine-checked. In the long run, I think executable, machine-checked, and a library that accumulates will win out, but only if it connects to data as well as SFT does, gets its laws right, and earns each theory's community's trust. So I expect convergence: methods like SFT-GMA becoming parts of a library like this one.

What is true here?

In mathematics, a proved theorem is true. Here there are only competing theories. What can a library build on?

Claim	Example	Status
A law entails a prediction	stability $\Rightarrow$ practiced items stay practiced	proved, permanent
A family has a law	discrimination nets have stability	proved, permanent
People have a law	people lack stability in recall	empirical, revisable

Which theory is right follows from the second and third rows together.



Part 8, what is true: conditional results, evidence, and limits.

Finally, part eight, and the question I promised. In mathematics, a proved theorem is true. Here we have competing theories, so what is true? There are three kinds of claim. That a law entails a prediction is proved, and permanent. That a family has a law is proved, and permanent. Whether people have a law is empirical, and revisable. Which theory is right follows from the second and third together, and the first two never expire, even if every model in the library is refuted.

Against the evidence

	Certainty	Stability	Order-free	Recency	Stored-only	Determinate
People (judged)	*lacks*	*mixed*	*lacks*	*has*	*mixed*	*mixed*

- A judgment, not a theorem, with sources, kept apart from the proofs.
- Each entry states its **bridge**: how a measure (a report) is taken to show a law (about retrieval).
- Activation fits every settled entry, but each fit rests on unformalized bridges.

Against the evidence

	Certainty	Stability	Order-free	Recency	Stored-only	Determinate
People (judged)	*lacks*	*mixed*	*lacks*	*has*	*mixed*	*mixed*

- A judgment, not a theorem, with sources, kept apart from the proofs.
- Each entry states its **bridge**: how a measure (a report) is taken to show a law (about retrieval).
- Activation fits every settled entry, but each fit rests on unformalized bridges.

So the matrix carries a row for people: a judgment, with sources, kept apart from the theorems. Each entry states its bridge, that is, how a measure, which is always a report, is taken to show something about retrieval. For example, endorsing an unseen prototype as old doesn't show you retrieved the prototype; a memory that retrieves a similar studied item would also call it old. As we've judged it, ACT-R fits every settled entry, but those fits are hypotheses until the bridges are formalized.

Where this is weaker than mathematics

- **Definitions are contestable.** A family is one formalization of a theory. We check that link with blind simulations and community implementations.
- **Laws are idealizations** and presuppose a representation (binary codes, feature-count similarity).
- **Families are flexible.** The informative facts are parameter regions, not single counterexamples.
- **Evidence is never final.** The empirical row will change; the proved rows won't, and the matrix says at once what follows.

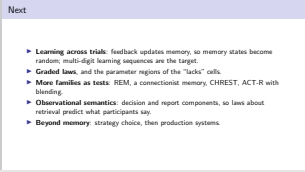
Where this is weaker than mathematics

- **Definitions are contestable.** A family is one formalization of a theory. We check that link with blind simulations and community implementations.
- **Laws are idealizations** and presuppose a representation (binary codes, feature-count similarity).
- **Families are flexible.** The informative facts are parameter regions, not single counterexamples.
- **Evidence is never final.** The empirical row will change; the proved rows won't, and the matrix says at once what follows.

Let me be clear about where this is weaker than mathematics. The definitions are contestable, which is why we do blind checks. The laws are idealizations, and they assume a representation. The families are flexible, so the most informative facts are about regions of their parameters, not single counterexamples. And the evidence is never final. But when it changes, the matrix tells you immediately what has been refuted.

Next

- **Learning across trials**: feedback updates memory, so memory states become random; multi-digit learning sequences are the target.
- **Graded laws**, and the parameter regions of the "lacks" cells.
- **More families as tests**: REM, a connectionist memory, CHREST, ACT-R with blending.
- **Observational semantics**: decision and report components, so laws about retrieval predict what participants say.
- **Beyond memory**: strategy choice, then production systems.

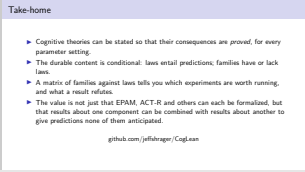


Next steps: learning across trials, where memory states themselves become random; graded laws, which will also tell us where in its parameters each family has each law; more families as tests, such as REM, a connectionist memory, and ACT-R with blending; decision and report components, so that laws about retrieval predict responses; and beyond memory, to strategy choice and production systems.

Take-home

- Cognitive theories can be stated so that their consequences are *proved*, for every parameter setting.
- The durable content is conditional: laws entail predictions; families have or lack laws.
- A matrix of families against laws tells you which experiments are worth running, and what a result refutes.
- The value is not just that EPAM, ACT-R and others can each be formalized, but that results about one component can be combined with results about another to give predictions none of them anticipated.

github.com/jeffshrager/CogLean



So, the take-home. Cognitive theories can be stated so that their consequences are proved, for every parameter setting. The durable content is conditional: laws entail predictions, and families have or lack laws. A matrix of families against laws tells you which experiments are worth running and what a result refutes. And the real value isn't that each theory can be formalized; it's that results about one component can be combined with results about another into predictions that no single model anticipated. Thank you.